

Road Network Planning and Navigation

Road Network Planning and Navigation

1. Course Content
2. Run Road Network Planning and Navigation Function
 - 2.1 Start Agent
 - 2.2 Start Sensors and Odometry
 - 2.3 Start Road Network Planning and Navigation Node
 - 2.4 Inter-Waypoint Navigation
 - 2.5 Navigation When Starting Position is Not on Road Network
3. Source Code Analysis
 - 3.1 route_bridge.py Analysis
 - 3.2 conver_speed.py Analysis
4. Navigation Parameter Debugging
 - 4.1 Navigation Controller Debugging
 - 4.2 Obstacle Avoidance and Costmap Debugging

1. Course Content

Basic: Use marked road network files for inter-road network navigation in the map

Advanced: Master debugging methods for navigation parameter files

2. Run Road Network Planning and Navigation Function

2.1 Start Agent

- Use the following command to start microROS agent to connect to the chassis

[!WARNING]

If you are using the `start_agent` command to start the microROS agent, you need to close the previous agent first. Note that this command is different from the previous microROS agent startup commands

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

- Modify ROS communication middleware DDS

[!WARNING]

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning and navigation chapter needs to use cycloneds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK & PI5 mainboards

```

131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
145

```

- ORIN mainboard

```

133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
137

```

```

:wq # save and exit
source ~/.bashrc # update environment

```

2.2 Start Sensors and Odometry

- Open a terminal on the host to start the vehicle's radar fusion node and odometry fusion
- (For PI5 users, start in the `m3` container)

```
ros2 launch m3_bringup car_base.launch.py
```

```

jetson@yahboom:~$ ros2 launch m3_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-11-19-18-43-09-279818-yahboom-1129502
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: =====Robot model ROSMASTER-M3 start=====
[INFO] [launch.user]: =====LASER_TYPE:single=====
[INFO] [launch.user]: LASER start sucefully!,Start by independent node
[INFO] [launch.user]: =====Robot Base Start=====
[INFO] [joint_state_publisher-1]: process started with pid [1129557]
[INFO] [robot_state_publisher-2]: process started with pid [1129559]
[INFO] [lasers_merger-3]: process started with pid [1129561]
[INFO] [scan_to_scan_filter_chain-4]: process started with pid [1129564]
[lasers_merger-3] [INFO] [1763548989.905313662] [laserscan_multi_merger]: Waiting for topics ...
[robot_state_publisher-2] [INFO] [1763548989.924917431] [robot_state_publisher]: got segment ascamera_hp60c_camera
link_0
[robot_state_publisher-2] [INFO] [1763548989.925142333] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1763548989.925167773] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1763548989.925181310] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1763548989.925191422] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-2] [INFO] [1763548989.925200830] [robot_state_publisher]: got segment laser23_frame
[robot_state_publisher-2] [INFO] [1763548989.925209982] [robot_state_publisher]: got segment lwheel1_Link
[robot_state_publisher-2] [INFO] [1763548989.925218911] [robot_state_publisher]: got segment lwheel2_Link
[robot_state_publisher-2] [INFO] [1763548989.925227999] [robot_state_publisher]: got segment rwheel1_Link
[robot_state_publisher-2] [INFO] [1763548989.925236735] [robot_state_publisher]: got segment rwheel2_Link
[scan_to_scan_filter_chain-4] [WARN] [1763548989.935781454] [rcl_logging_rosout]: Publisher already registered for
provided node name. If this is due to multiple nodes with the same name then all logs for that logger name will g
o out over the existing publisher. As soon as any node with that name is destructed it will unregister the publish
er, preventing any further logs for that name from being published on the rosout topic.
[joint_state_publisher-1] [INFO] [1763548990.531349805] [joint_state_publisher]: Waiting for robot_description to
be published on the robot_description topic...
[lasers_merger-3] [INFO] [1763548990.907903719] [laserscan_multi_merger]: Subscribing to topics 2
[scan_to_scan_filter_chain-4] [WARN] [1763548990.924837083] [laser_filter]: diagnostic_updater: No HW_ID was set.
This is probably a bug. Please report it. For devices that do not have a HW_ID, set this value to 'none'. This war
ning only occurs once all diagnostics are OK. It is okay to wait until the device is open before calling setHardwa
reID.
[INFO] [launch.user]: =====Merge_Odom Localization Start=====
[INFO] [ekf_node-5]: process started with pid [1129717]

```

2.3 Start Road Network Planning and Navigation Node

- If the ros2_kilted docker container is not started, you need to start the container first

```
bringup_ros2kilted
```

- Open a terminal inside the container

```
exec_ros2kilted
```

- (PI5 & RDK mainboards) Visualization is started on the accompanying virtual machine
`Rosmaster-RoadNet-Ubuntu24.04`,

```
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/nav2_view.rviz
```

- Start road network planning and navigation node

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

[!TIP]

Note: If you previously saved a custom map name, you need to modify the default startup parameter's map name.

Optional startup parameter description

Example: `ros2 launch road_net_route roadnet_nav2.launch.py map:=/xxx/test.yaml
pose_map:=/xxx/test roadnet_file:=/xxx/test.geojson`

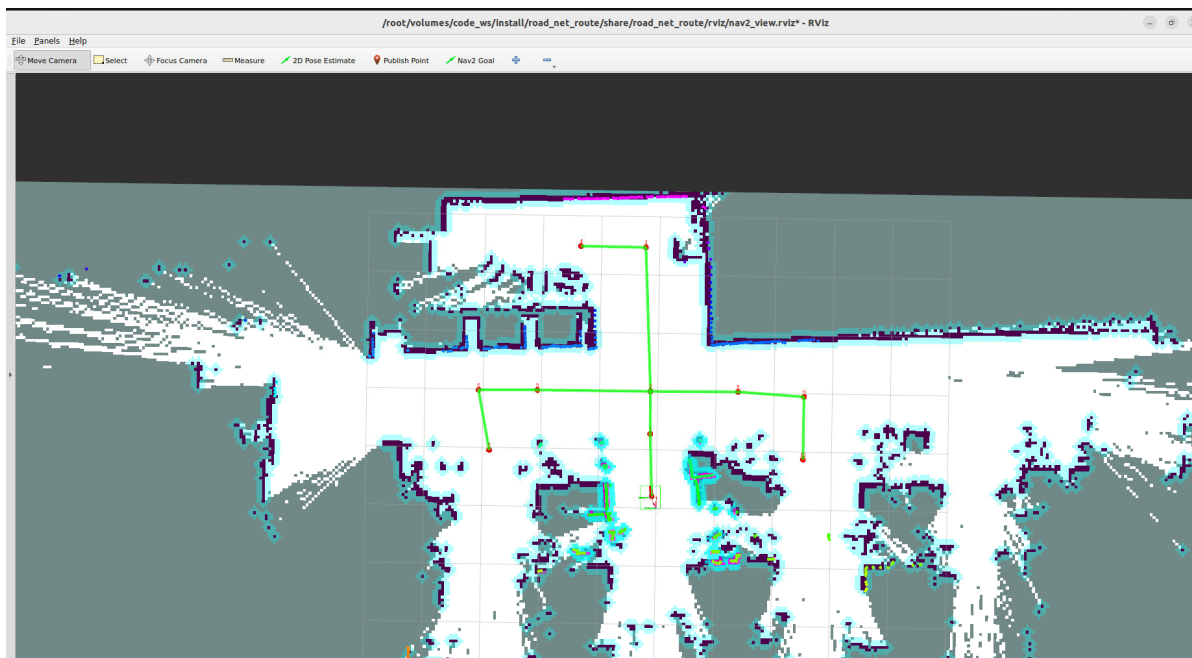
map The grid map file name to be loaded, default value is `map.yaml`

pose_map The pose map file name to be loaded. When the navigation global positioning method uses slam_toolbox pure localization, the pose map must be passed, default value is `map`

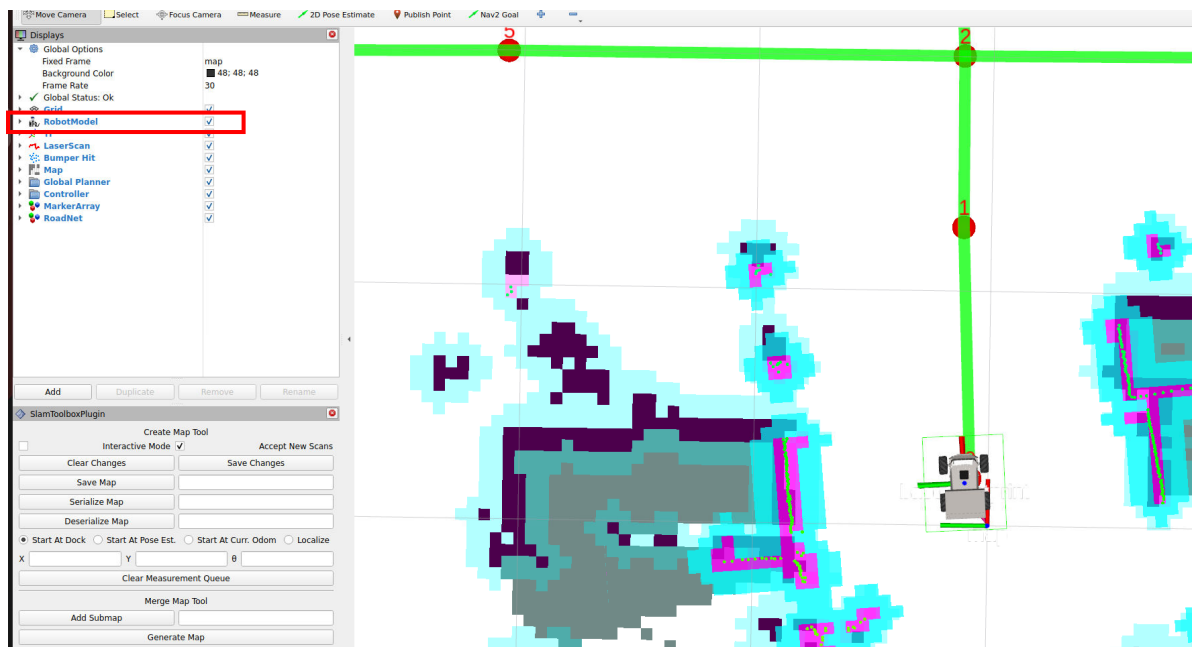
params_file Navigation parameter file, default value is
`os.path.join(get_package_share_directory('road_net_route'), 'config',
'nav2_kilted.yaml')`

roadnet_file Road network description file, default value is `map.geojson`

localization_mode Global positioning method, default is `slam_toolbox`



- To improve RViz startup speed, the robot model is not loaded by default. If you need to display the robot model, check RobotModel in the left tab.



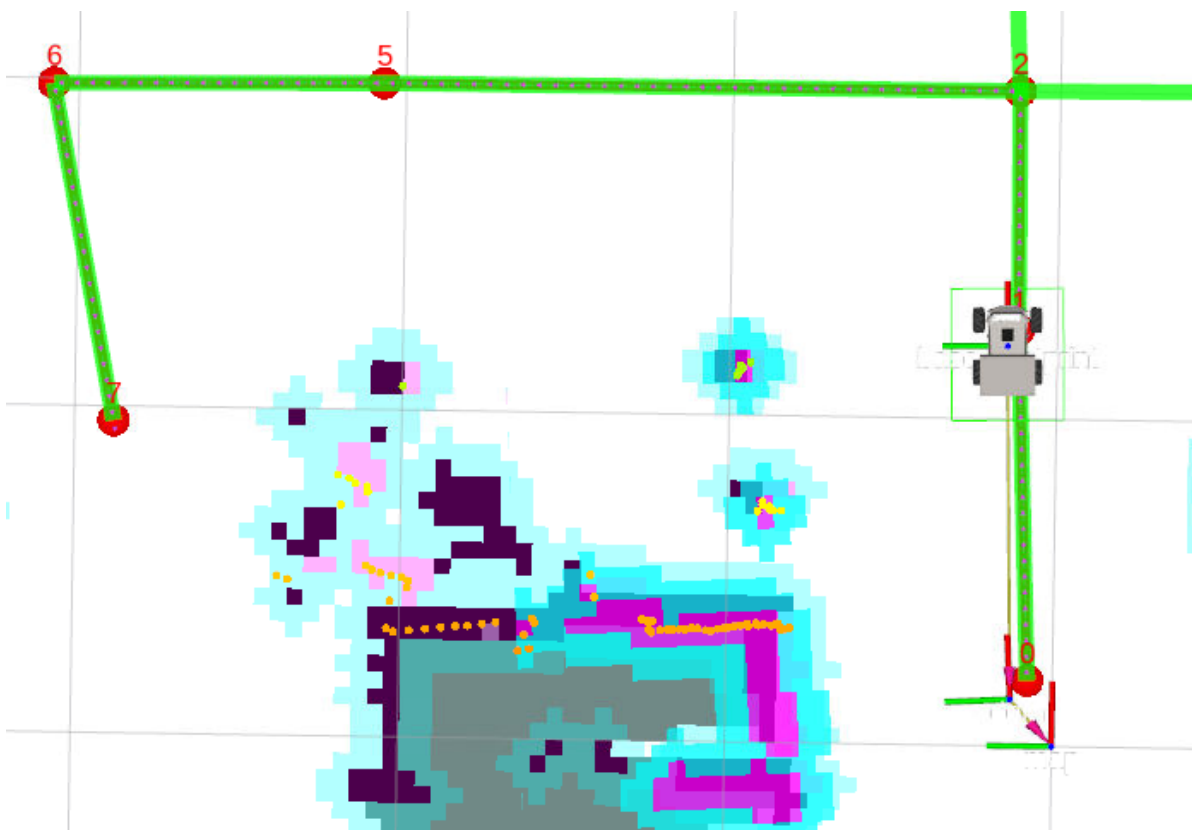
2.4 Inter-Waypoint Navigation

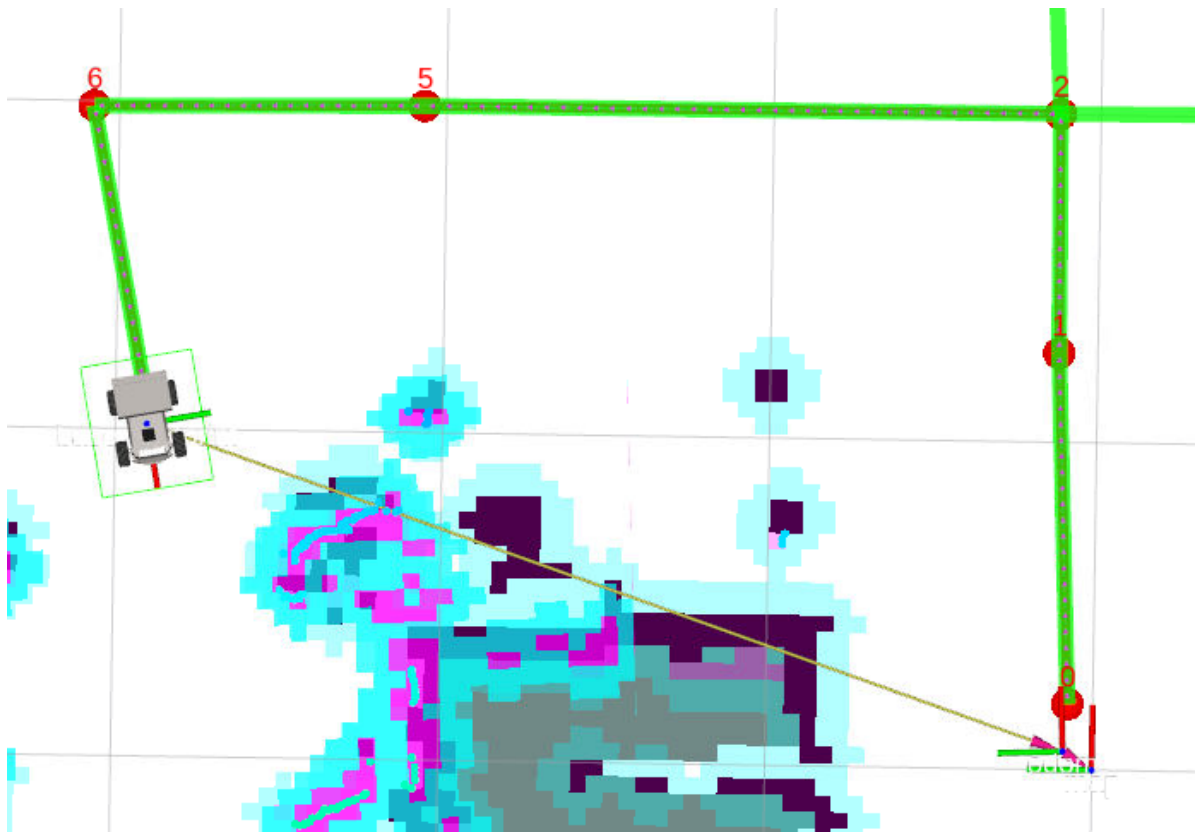
- Here we use road network navigation by publishing the `/road_net_nav_id` topic in the terminal. For example, in the image, our robot vehicle is near waypoint 0, and we need to navigate to waypoint 7

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [0, 7]}"
```

```
jetson@yahboom: ~ 67x20
jetson@yahboom:~$ ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [0, 7]}"
```

- Then you can see in RViz that the planned global path is only planned within the road network

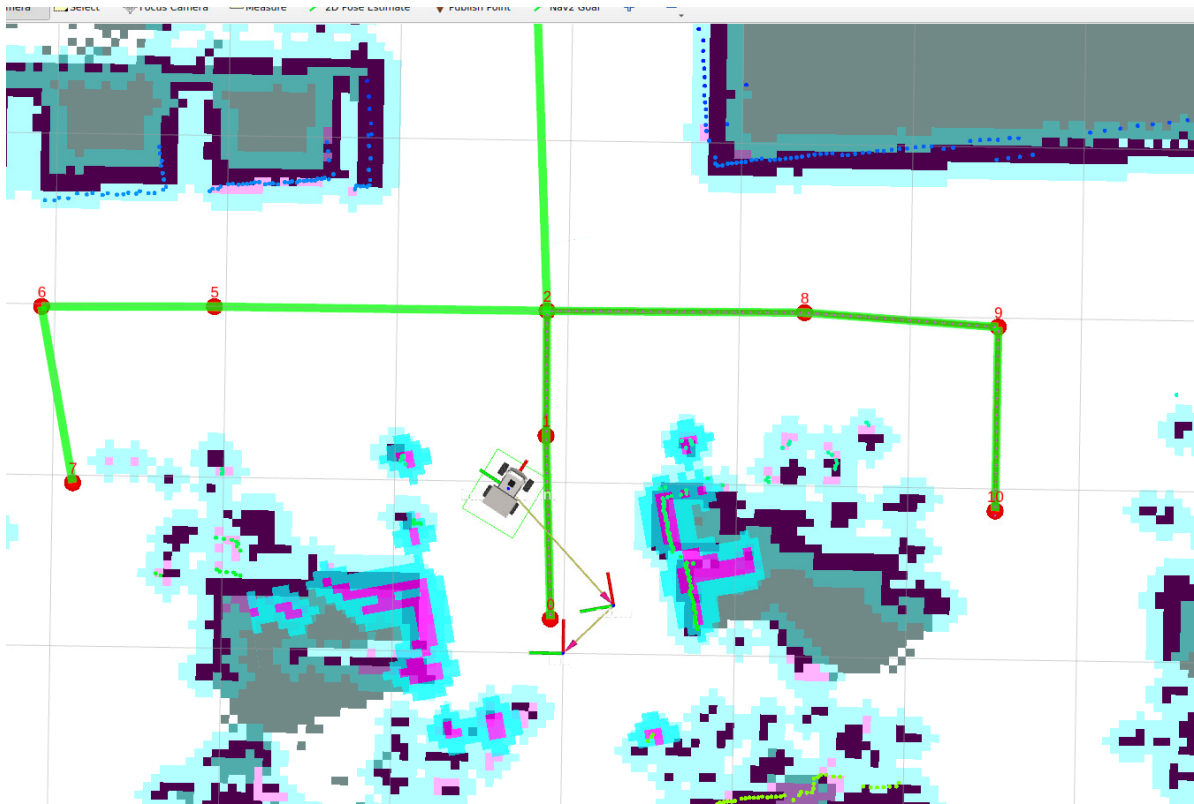
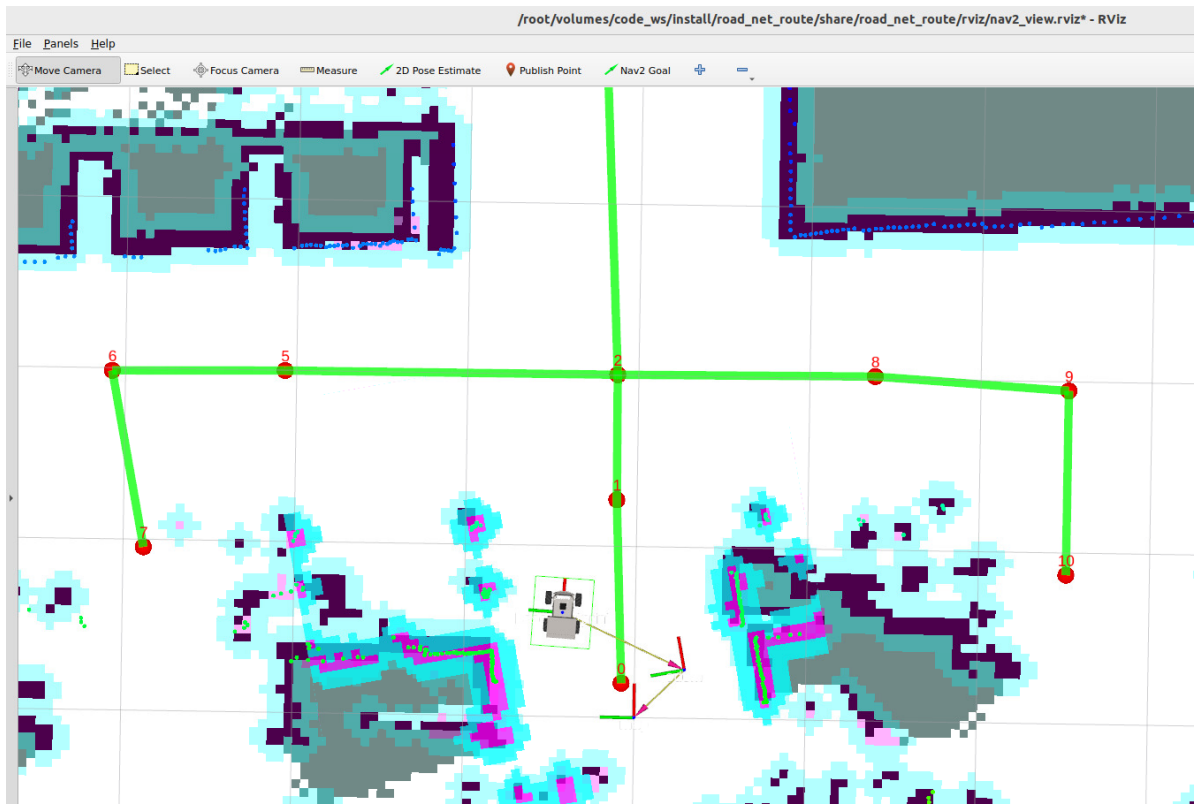


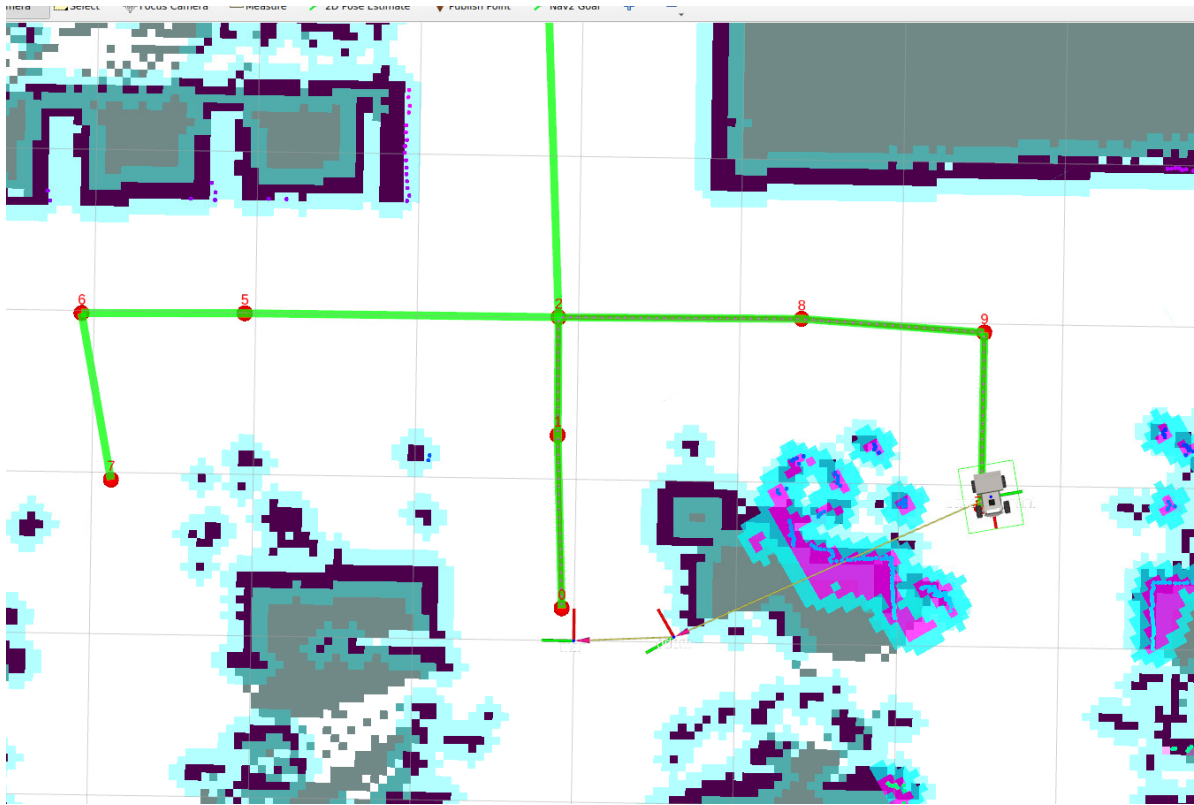


2.5 Navigation When Starting Position is Not on Road Network

- If the starting position is not on a road network point or edge, when executing road network navigation, the robot vehicle will automatically first enter the nearest road network edge, then move along the road network. For example, in the following situation, the vehicle is not in the road network, we observe that the nearest waypoint is 0, here we plan to go to waypoint 10, then publish the start and end point topics

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [0, 10]}"
```



3. Source Code Analysis

- Implementation program source code path:

```
~/yahboomM3_ws/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

```
~/yahboomM3_ws/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/convert_speed.py
```

3.1 route_bridge.py Analysis

- In the initialization function of the Route_Bridge class, a `road_net_nav_id` topic subscriber is created to receive `Int16MultiArray` data from the `road_net_nav_id` topic (format is `[start_id, goal_id]`), directly calling `getAndTrackRoute` to initiate road network navigation:
- Input validation: If data length is not 2, report error; if it is `[-1, -1]`, cancel navigation.
- Navigation execution: Call `self.navigators.getAndTrackRoute(start_id, goal_id)`, return navigation task handle.
- Feedback processing: Loop to get navigation feedback (`feedback`), monitor node switching (`last_node_id / next_node_id`) and rerouting (`rerouted`) events, if rerouted then call `followPath` to update local path.
- Result processing: Output logs according to `TaskResult` (success/cancelled/failed), and publish results through `/action_feedback`.

```
self.road_net_nav_ID_sub= self.create_subscription(Int16MultiArray,
"road_net_nav_id", self.id_nav_callback, 10)
```

```

def id_nav_callback(self, msg: Int16MultiArray):
    ''' Callback function for navigation route messages using IDs '''
    if len(msg.data) != 2:
        self.get_logger().error(Fore.RED+"Invalid ID navigation message received."+Fore.RESET)
        return
    if msg.data[0]==-1 and msg.data[1]==-1 :
        self.navigator.cancelTask()
        self.get_logger().info(Fore.YELLOW+"navigation task canceled via message."+Fore.RESET)
        return
    route_tracking_task = self.navigator.getAndTrackRoute(msg.data[0],
msg.data[1])
    task_canceled = False
    last_feedback = None
    follow_path_task = RunningTask.NONE
    while not self.navigator.isTaskComplete(task=route_tracking_task):
        feedback = self.navigator.getFeedback(task=route_tracking_task)
        while feedback is not None:
            if not last_feedback or (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                self.get_logger().info('Passed node ' +
str(feedback.last_node_id) +
                ' to next node ' + str(feedback.next_node_id) +
                ' along edge ' + str(feedback.current_edge_id) + '.')
            last_feedback = feedback
            if feedback.rerouted: # or follow_path_task == RunningTask.NONE
                self.get_logger().info('Passing new route to controller!')
                follow_path_task = self.navigator.followPath(feedback.path)
                feedback = self.navigator.getFeedback(task=route_tracking_task)
            if self.navigator.isTaskComplete(task=follow_path_task):
                self.get_logger().info('Controller or waypoint follower server
completed its task!')
                self.navigator.cancelTask()
                task_canceled = True
            while not self.navigator.isTaskComplete(task=follow_path_task) and not
task_canceled:
                time.sleep(0.1)

        result = self.navigator.getResult()
        if result == TaskResult.SUCCEEDED:
            self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

            self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
            elif result == TaskResult.CANCELED:
                self.get_logger().info(Fore.YELLOW+"navigation task
canceled."+Fore.RESET)
            elif result == TaskResult.FAILED:
                self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)
                self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
            else:
                self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)
                self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

```

3.2 conver_speed.py Analysis

- Since the navigation2 in the ros2 kilted version publishes speed topics in `TwistStamped` format, while the ROSMASTER-M3 chassis microROS node accepts Twist format speed topics for chassis control, the two cannot communicate directly. A bridge is needed for speed message format conversion

We can use the following commands to view the difference between the two message types. The main difference is `std_msgs/Header header`

- `TwistStamped` format speed topic

```
ros2 interface show geometry_msgs/msg/TwistStamped
```

```
# A twist with reference coordinate frame and timestamp
```

```
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
Twist twist
  Vector3 linear
    float64 x
    float64 y
    float64 z
  Vector3 angular
    float64 x
    float64 y
    float64 z
```

- `Twist` format speed topic

```
ros2 interface show geometry_msgs/msg/Twist
```

```
# This expresses velocity in free space broken into its linear and angular parts.
```

```
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

- Here in the initialization function of the `TwistStampedToTwist` class, a speed topic subscriber in `TwistStamped` format is defined, and then in the callback function, the speed in

TwistStamped format is converted to Twist format speed topic and published.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped, Twist
from colorama import Fore
class TwistStampedToTwist(Node):
    def __init__(self):
        super().__init__('conver_speed')
        self.sub = self.create_subscription(
            TwistStamped,
            '/cmd_vel',
            self.callback,
            5
        )
        self.pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            5
        )
        self.get_logger().info(Fore.GREEN + "TwistStamped → Twist The converter
has been started."+Fore.RESET)

    def callback(self, msg:TwistStamped):
        t = Twist()
        t.linear = msg.twist.linear
        t.angular = msg.twist.angular
        self.pub.publish(t)

def main():
    rclpy.init()
    node = TwistStampedToTwist()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

4. Navigation Parameter Debugging

Road network navigation parameter file path:

```
~/yahboomM3_ws/ros2_kilted/volumes/code_ws/src/road_net_route/config/nav2_kilted
.yaml
```

4.1 Navigation Controller Debugging

[!TIP]

Parameter Adjustment Guide

- If you need to adjust movement speed and rotational angular velocity during navigation, adjust `desired_linear_vel` and `max_angular_accel`
- If you need to adjust accuracy of reaching target points, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`
- For other parameter adjustments, see the comments below

```
controller_server:
  ros__parameters:
    # ===== Controller Core Global Parameters
    =====
    # Controller main loop frequency (Hz), determines local path planning and
    # speed command publishing frequency, needs to match hardware execution frequency
    controller_frequency: 20.0
    # Costmap update timeout (seconds), if costmap is not updated within this
    # time, controller will stop outputting speed commands to avoid planning based on
    # outdated map
    costmap_update_timeout: 0.30
    # X direction (forward/backward) minimum velocity threshold (meters/second),
    # speed commands below this value will be determined as invalid, used to filter
    # tiny jitter
    min_x_velocity_threshold: 0.001
    # Y direction (lateral) minimum velocity threshold (meters/second), only
    # applicable to omnidirectional mobile robots (such as mecanum wheels),
    # differential drive robots can be set to larger value (such as 0.5) to disable
    # lateral movement
    min_y_velocity_threshold: 0.5
    # Rotation direction minimum velocity threshold (radians/second), rotation
    # commands below this value will be determined as invalid, filtering tiny rotation
    # jitter
    min_theta_velocity_threshold: 0.001
    # Controller failure tolerance (meters), when local planning path deviation
    # exceeds this value, determines controller execution failure, triggers replanning
    failure_tolerance: 0.3
    # Progress checker plugin list, used to monitor if robot is moving towards
    # target
    progress_checker_plugins: ["progress_checker"]
    # Goal checker plugin list, used to determine if robot has reached target
    # pose
    goal_checker_plugins: ["general_goal_checker"]
    # Controller plugin list, core is local path tracking algorithm, here uses
    # pure pursuit controller corresponding to FollowPath
    controller_plugins: ["FollowPath"]
    # Whether to use real-time priority scheduling for controller threads
    use_realtime_priority: false
    # Speed limit topic name, controller will subscribe to speed commands on this
    # topic, dynamically adjust maximum linear/angular velocity (such as speed
    # reduction during obstacle avoidance)
    speed_limit_topic: "speed_limit"

    # ===== Progress Checker Parameters (SimpleProgressChecker)
    =====
```

```

progress_checker:
    # Progress checker plugin type, SimpleProgressChecker is basic
    implementation, monitors robot movement distance and time
    plugin: "nav2_controller::SimpleProgressChecker"
    # Minimum distance threshold robot needs to move (meters), if robot does
    not move more than this distance within movement_time_allowance time, determined
    as no progress
    required_movement_radius: 0.05
    # Maximum allowed no progress time (seconds), if required_movement_radius
    is still not met after this time, controller will abort task (error code 105)
    movement_time_allowance: 10.0

# ===== Goal Checker Parameters (SimpleGoalChecker)
=====
general_goal_checker:
    # Whether to be a stateful checker, when True will continuously monitor
    target pose until tolerance is met; False only performs single check
    stateful: True
    # Goal checker plugin type, SimpleGoalChecker is basic implementation,
    determines if position and angle are within tolerance range
    plugin: "nav2_controller::SimpleGoalChecker"
    # XY plane position tolerance (meters), when deviation between robot
    position and target position is less than this value, determines position
    reached
    xy_goal_tolerance: 0.08
    # Heading angle tolerance (radians), when deviation between robot heading
    and target heading is less than this value, determines angle reached
    yaw_goal_tolerance: 0.03

# ===== Pure Pursuit Controller Parameters
(RegulatedPurePursuitController) =====
FollowPath:
    # Local path tracking plugin type, RegulatedPurePursuitController is a
    pure pursuit algorithm with speed adjustment, suitable for differential drive
    robots
    plugin:
"nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
    # Desired linear velocity (meters/second), target linear speed during
    normal robot operation, will be dynamically scaled by obstacle
    avoidance/curvature adjustment
    desired_linear_vel: 0.2
    # Lookahead distance (meters), core parameter of pure pursuit algorithm,
    determines distance of path points the robot tracks, needs to match robot speed
    lookahead_dist: 0.5
    # Minimum lookahead distance (meters), prevents lookahead distance from
    being too small causing frequent robot turning
    min_lookahead_dist: 0.05
    # Maximum lookahead distance (meters), prevents lookahead distance from
    being too large causing robot slow response
    max_lookahead_dist: 0.9
    # Lookahead time (seconds), if use_velocity_scaled_lookahead_dist is
    enabled, lookahead distance = desired_linear_vel * lookahead_time
    lookahead_time: 1.5
    # Angular velocity for rotating to target heading (radians/second),
    rotation speed for adjusting heading after robot reaches position
    rotate_to_heading_angular_vel: 1.3
    # TF transformation tolerance (seconds), allows maximum deviation between
    robot pose transformation timestamp and current time, solves TF delay issues

```

```
transform_tolerance: 0.5
# Whether to use velocity scaled lookahead distance, when True, lookahead
distance changes dynamically with robot speed (faster speed means farther
lookahead)
use_velocity_scaled_lookahead_dist: false
# Minimum linear velocity when approaching target (meters/second), when
robot is close to target, reduce speed to this value to avoid overshooting
target
min_approach_linear_velocity: 0.05
# Approach velocity scaling distance (meters), when robot distance to
target is less than this value, start linear speed reduction (from
desired_linear_vel to min_approach_linear_velocity)
approach_velocity_scaling_dist: 1.0
# Whether to enable collision detection, when True, controller will check
for collisions on lookahead path to avoid robot hitting obstacles
use_collision_detection: false
# Maximum allowed collision time to lookahead point (seconds), only
effective when use_collision_detection is True, used to determine collision risk
max_allowed_time_to_collision_up_to_carrot: 1.0
# Whether to enable path curvature-based linear velocity adjustment, when
True, smaller turning radius means lower linear velocity
use_regulated_linear_velocity_scaling: true
# Whether to enable cost map-based linear velocity adjustment, when True,
closer obstacles mean lower linear velocity
use_cost_regulated_linear_velocity_scaling: false
# Minimum turning radius to trigger curvature adjustment (meters), when
turning radius corresponding to path curvature is less than this value, start
speed reduction
regulated_linear_scaling_min_radius: 0.05
# Minimum speed for curvature adjustment (meters/second), even with
extremely small turning radius, linear velocity will not be lower than this
value
regulated_linear_scaling_min_speed: 0.25
# Whether to use fixed curvature lookahead distance, when True, lookahead
distance is determined by curvature_lookahead_dist
use_fixed_curvature_lookahead: false
# Curvature lookahead distance (meters), only effective when
use_fixed_curvature_lookahead is True
curvature_lookahead_dist: 1.0
# Whether to enable "rotate to heading first then move", when True, if
robot heading deviates too much from path direction, rotate first then move
forward
use_rotate_to_heading: true
# Minimum angle to trigger rotate to heading (radians), when robot heading
deviation from path direction exceeds this value, start rotation adjustment
rotate_to_heading_min_angle: 0.785 # approximately 45 degrees
# Maximum angular acceleration (radians/second2), limits robot rotation
acceleration to avoid hardware damage from sudden turns
max_angular_accel: 1.2
# Maximum distance for robot pose search (meters), when controller cannot
find matching robot pose on path, search for path points within this distance
max_robot_pose_search_dist: 10.0
# Whether to interpolate curvature after goal point, when True, path after
goal point will interpolate curvature to avoid robot sudden stop after reaching
goal
interpolate_curvature_after_goal: false
```



```

# Cost scaling distance (meters), only effective when
use_cost_regulated_linear_velocity_scaling is True, used to calculate weight for
obstacle avoidance speed reduction
cost_scaling_dist: 0.08
# Cost scaling gain, only effective when
use_cost_regulated_linear_velocity_scaling is True, larger gain means more
obvious obstacle avoidance speed reduction
cost_scaling_gain: 1.0
# Inflation cost scaling factor, used to convert inflation layer cost of
costmap to speed scaling coefficient, larger value means more sensitive to
inflation area obstacle avoidance
inflation_cost_scaling_factor: 3.0

```

4.2 Obstacle Avoidance and Costmap Debugging

[!TIP]

- If the robot navigates too close to obstacles in the map, you can make the robot stay away from obstacles by adjusting the inflation area. Increase the `inflation_radius` and `cost_scaling_factor` in `local_costmap` and `global_costmap` (the larger the scaling factor value, the less likely the robot will approach obstacles)
- For other parameter adjustments, see the comments below

```

# Local costmap configuration: Short-term costmap for real-time robot obstacle
avoidance, scrolls and updates with robot movement
local_costmap:
  local_costmap:
    ros__parameters:
      # ===== Costmap Core Global Parameters
      =====
      # Costmap update frequency (Hz), determines local map refresh speed, needs
to match sensor frequency
      update_frequency: 5.0
      # Costmap publish frequency (Hz), frequency of publishing map to
visualization tools like RViz, lower than update frequency to reduce bandwidth
usage
      publish_frequency: 2.0
      # Global coordinate frame of costmap, local map uses odom frame
      global_frame: odom
      # Robot base coordinate frame, consistent with hardware-configured robot
center coordinate frame
      robot_base_frame: base_link
      # Whether to enable rolling window mode, core feature of local map: map
scrolls with robot movement, only retains map around robot area
      rolling_window: true
      # Rolling window width (meters), horizontal range of local map
      width: 3
      # Rolling window height (meters), vertical range of local map
      height: 3
      # Costmap resolution (meters/pixel), represents actual distance per pixel,
0.05 means 5 centimeters/pixel
      resolution: 0.05

```

```

# Robot footprint (polygon), represents actual robot shape with coordinate
list
footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
# Robot footprint padding distance (meters), additional safety distance
extended outside actual footprint for obstacle avoidance buffering
footprint_padding: 0.0616
# Costmap layer plugin list, loaded in order (voxel_layer: voxel obstacle
layer; inflation_layer: inflation layer)
plugins: ["voxel_layer", "inflation_layer"]
# Costmap filter plugin list, used for map post-processing
(keepout_filter: restricted area filtering)
filters: ["keepout_filter"]
# Whether to always publish complete costmap, when True publishes full map
on each update, False only publishes changed areas (reduces bandwidth)
always_send_full_costmap: True
# Service introspection mode, disabled means disable service
introspection, options are enabled/verbose
service_introspection_mode: "disabled"

# ===== Filter Plugin: Restricted Area Filtering
(KeepoutFilter) =====
keepout_filter:
# Filter plugin type, KeepoutFilter used to mark restricted areas in
costmap
plugin: "nav2_costmap_2d::KeepoutFilter"
# Whether to enable restricted area filtering, KEEPOUT_ZONE_ENABLED is
external parameter (needs to be assigned true/false at startup)
enabled: KEEPOUT_ZONE_ENABLED
# Subscription topic for restricted area information, receives geometry
and position information of restricted areas (such as PolygonStamped)
filter_info_topic: "keepout_costmap_filter_info"
# Whether to override lethal cost of restricted areas, when True uses
custom cost to replace default lethal cost (255)
override_lethal_cost: True
# Custom lethal cost for restricted areas, 200 means lower than default
lethal cost (255) but still high-cost restricted area
lethal_override_cost: 200

# ===== Layer Plugin: Inflation Layer (InflationLayer)
=====
inflation_layer:
# Layer plugin type, InflationLayer used to generate inflation areas
around obstacles to avoid robot collisions
plugin: "nav2_costmap_2d::InflationLayer"
# Cost scaling factor, controls cost decay speed in inflation areas
(larger value means faster cost decay, smaller inflation area)
cost_scaling_factor: 3.0
# Inflation radius (meters), inflation range around obstacles, minimum
safe distance from robot footprint to obstacles
inflation_radius: 0.1

# Global costmap configuration: Long-term costmap for robot global path planning,
covers entire environment
global_costmap:
global_costmap:
ros__parameters:
# ===== Costmap Core Global Parameters
=====

```

```

# Costmap update frequency (Hz), global map does not need high-frequency
updates, 1-2Hz is sufficient
update_frequency: 1.0
# Costmap publish frequency (Hz), can be consistent with update frequency
publish_frequency: 1.0
# Global coordinate frame of costmap, global map uses map frame (globally
consistent, suitable for path planning)
global_frame: map
# Robot base coordinate frame, consistent with local map
robot_base_frame: base_link
# Robot radius (replaced by footprint, commented out)
# robot_radius: 0.22
# Robot footprint (consistent with local map)
footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
# Robot footprint padding distance (consistent with local map)
footprint_padding: 0.0616
# Costmap resolution (consistent with local map to ensure coordinate
matching)
resolution: 0.05
# Whether to track unknown space, when True marks unexplored areas in map
(used for SLAM and unknown environment planning)
track_unknown_space: true
# Costmap layer plugin list (static_layer: static layer; obstacle_layer:
2D obstacle layer; inflation_layer: inflation layer)
plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
# Costmap filter plugin list (keepout_filter: restricted area filtering;
speed_filter: speed limit filtering)
filters: ["keepout_filter", "speed_filter"]
# Whether to always publish complete costmap
always_send_full_costmap: True
# Service introspection mode
service_introspection_mode: "disabled"

inflation_layer:
# Layer plugin type, InflationLayer used to generate inflation areas
around obstacles to avoid robot collisions
plugin: "nav2_costmap_2d::InflationLayer"
# Cost scaling factor, controls cost decay speed in inflation areas
(larger value means faster cost decay, smaller inflation area)
cost_scaling_factor: 3.0
# Inflation radius (meters), inflation range around obstacles, minimum
safe distance from robot footprint to obstacles
inflation_radius: 0.1

```